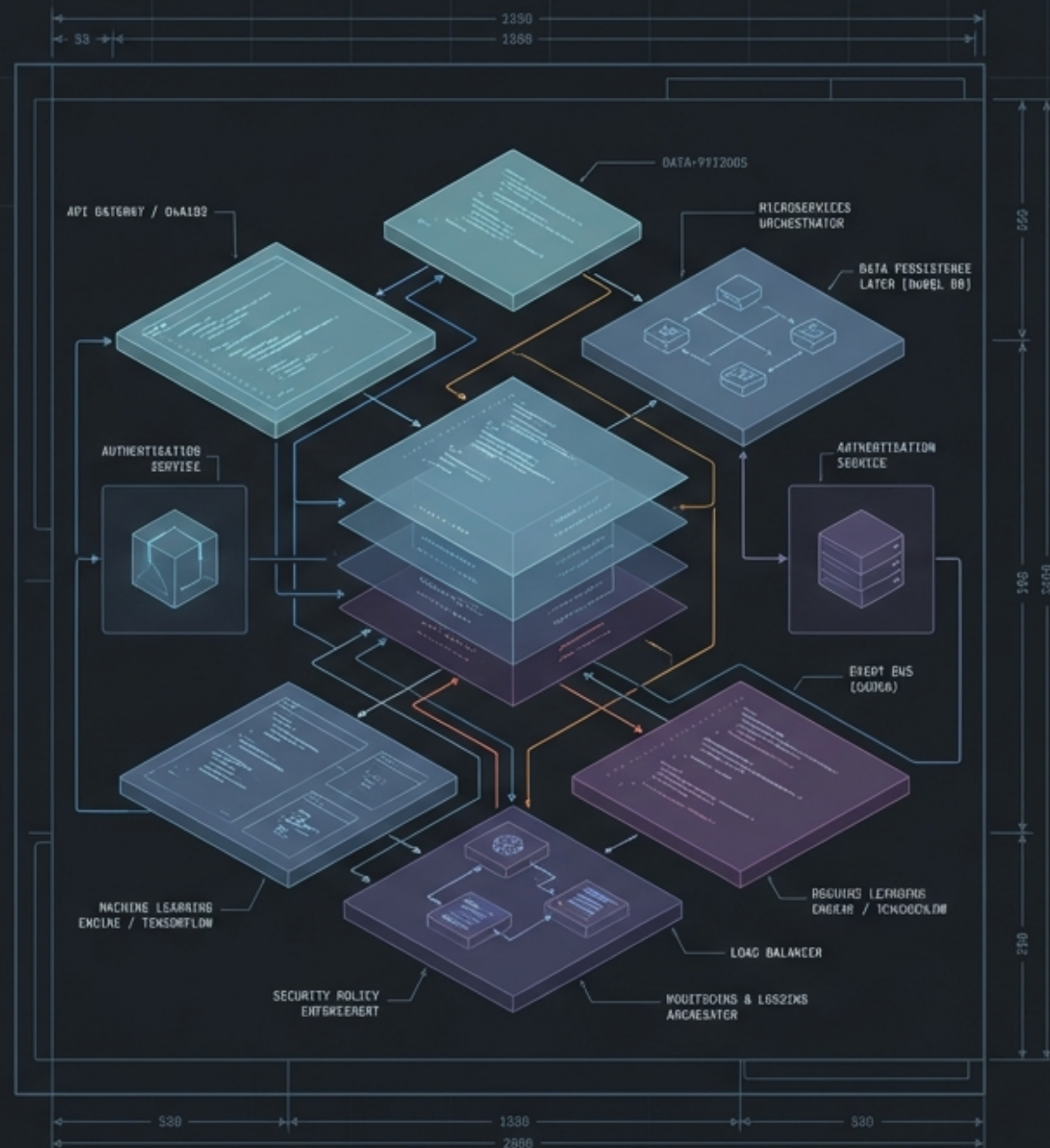


The Professional Operating System

Architecture of an ICT Project:
Navigating the 19 Domains of Control



Code Compiles. Projects Crash.

The Code

```
def calculate_efficiency(resources: int, time: float) -> float:
    """Calculates project efficiency based on resource input and time."""
    if time <= 0:
        raise ValueError("Time must be positive.")
    try:
        efficiency = resources / time
        return round(efficiency, 2)
    except Exception as e:
        log_error(e)
        return 0.0

class ProjectManager:
    def __init__(self, team_size: int, budget: float):
        self.team_size = team_size
        self.budget = budget
        self.status = "Initiated"

    def execute_phase(self, phase_name: str, duration: int):
        print(f"Executing {phase_name} for {duration} weeks.")
        # Logic proceeds predictably...
        self.status = f"{phase_name} Complete"
        return True
```

As a developer, your primary constraint is logic.

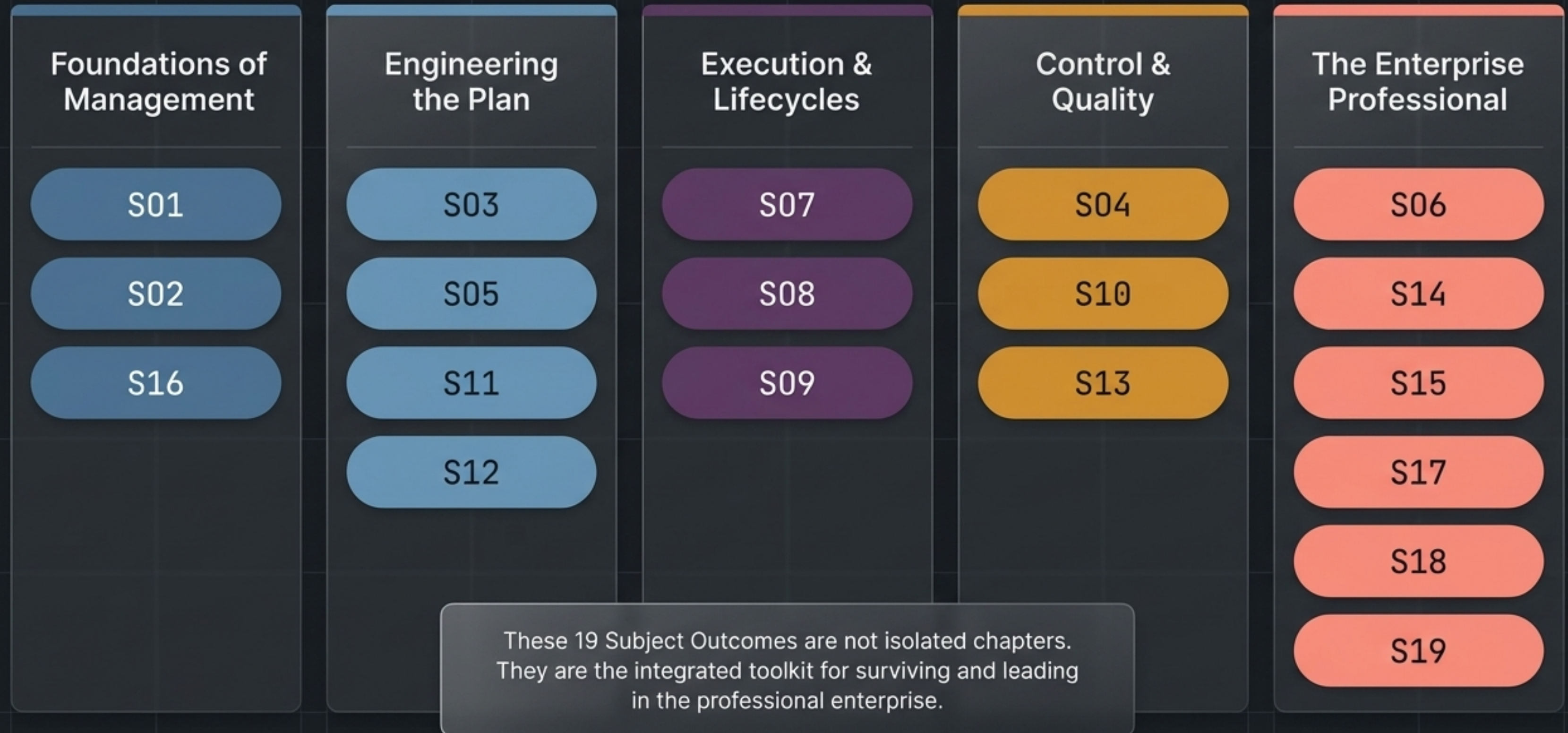
The Project



As an ICT professional, your constraints are time, capital, and people. Software projects fail at an alarming rate due to uncontrolled scope creep, mathematical errors in estimation, and human friction.

Takeaway: Technical competence is no longer enough. You need an architecture for control.

The Architecture of Control: 19 Outcomes, 5 Domains



Domain 1: Defining the Endeavour

The Operations vs. Projects Matrix (SO1)



Operations (Ongoing & Repetitive)

Duration: Continuous pipeline of work.

Output: Maintains the status quo; repetitive services (e.g., standard IT support).

Evolution: Standardized and static.



Projects (Temporary & Unique)

Duration: Definite beginning and end. Terminates when objectives are met.

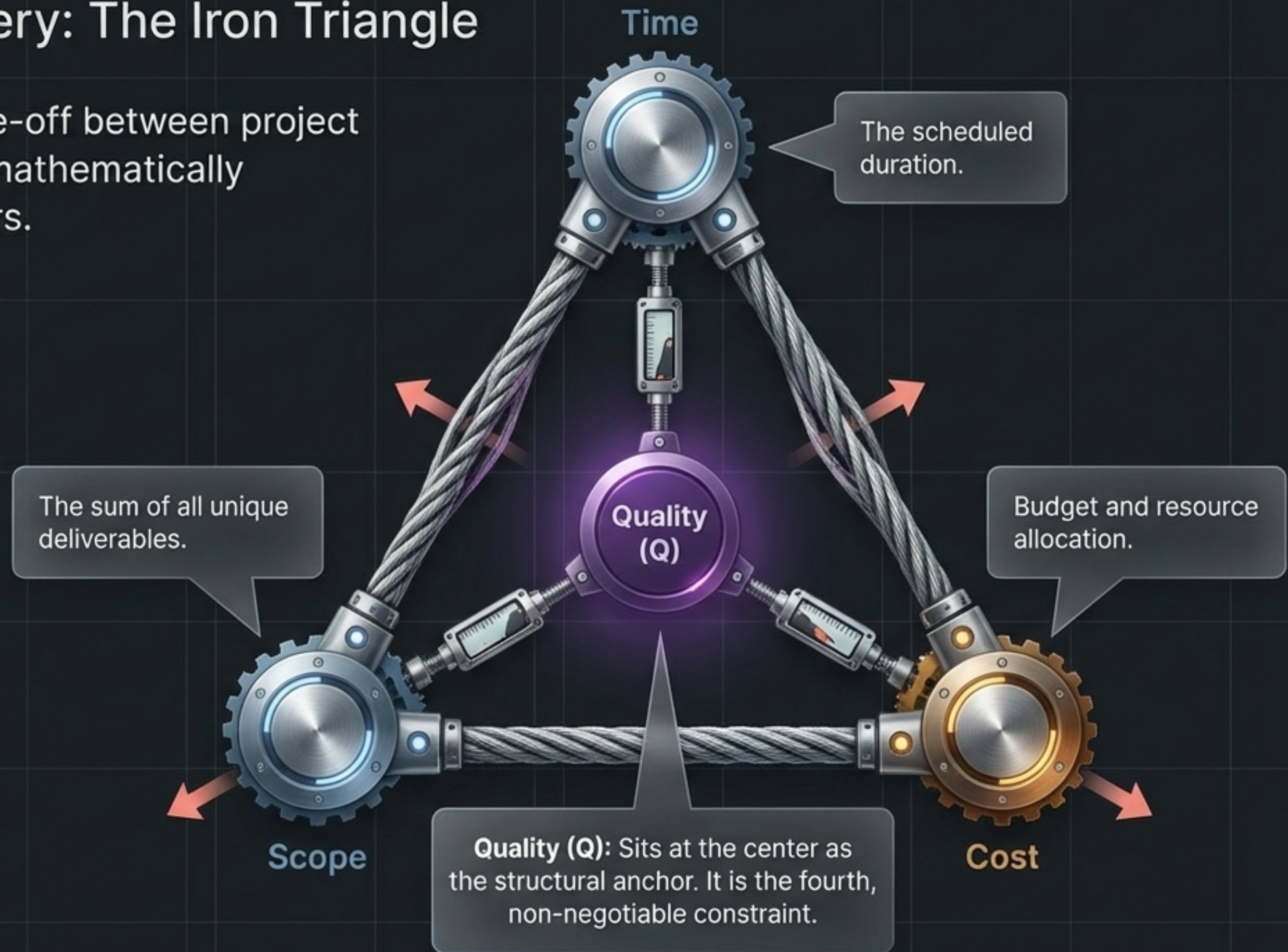
Output: Creates a unique product, capability, or result.

Evolution: Progressive Elaboration (broad early, explicit later).

Note: Temporary does NOT mean short. It means finite.

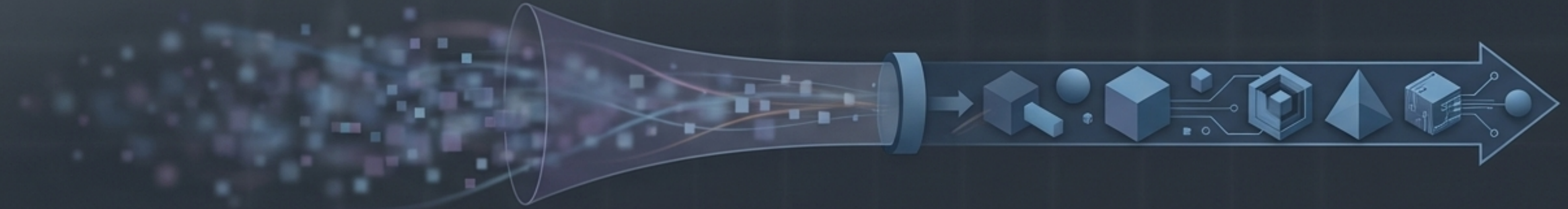
The Physics of Delivery: The Iron Triangle

There is a fundamental trade-off between project constraints. Changing one mathematically forces a change in the others.



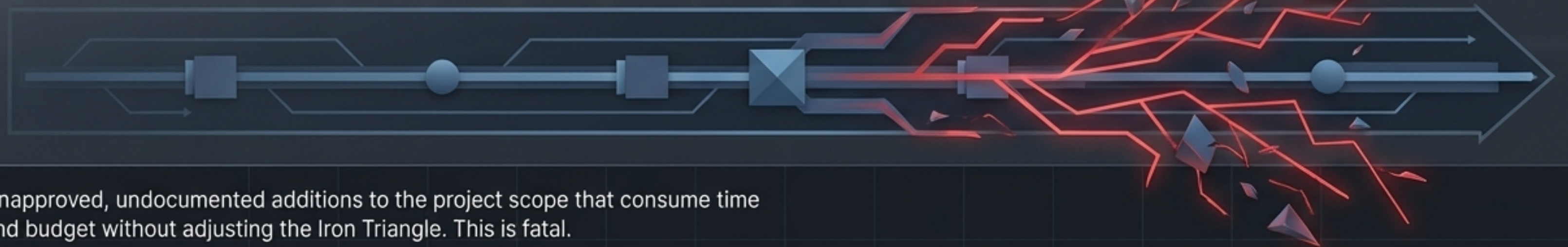
Evolution vs. Mutation

Progressive Elaboration (Controlled)



Developing in steps, continuing by increments. Scope is broad early, becoming meticulously explicit as understanding grows. This is planned.

Scope Creep (Uncontrolled)



Unapproved, undocumented additions to the project scope that consume time and budget without adjusting the Iron Triangle. This is fatal.

The Management Mindset Matrix (S02)

Henri Fayol: The Classical Architect

- **Focus:** Structure and rigid hierarchy (14 Principles).
- **Key Concept:** Unity of Command (Every employee has one, and only one, boss).
- **Key Concept:** Remuneration (Fair pay) vs. Equity (Equal treatment).

Henry Mintzberg: The Dynamic Operator

- **Focus:** What managers actually do (10 Roles across 3 Categories).
- **Information Roles (~40% of time):** Monitor (receives intel) vs. Disseminator (shares intel).
- **Decision Roles:** Entrepreneur (initiates change) vs. Disturbance Handler (reacts to external pressure).

The Two-Boss Dilemma

Structural realities in modern ICT environments (SO2 / SO6)

Functional Org:

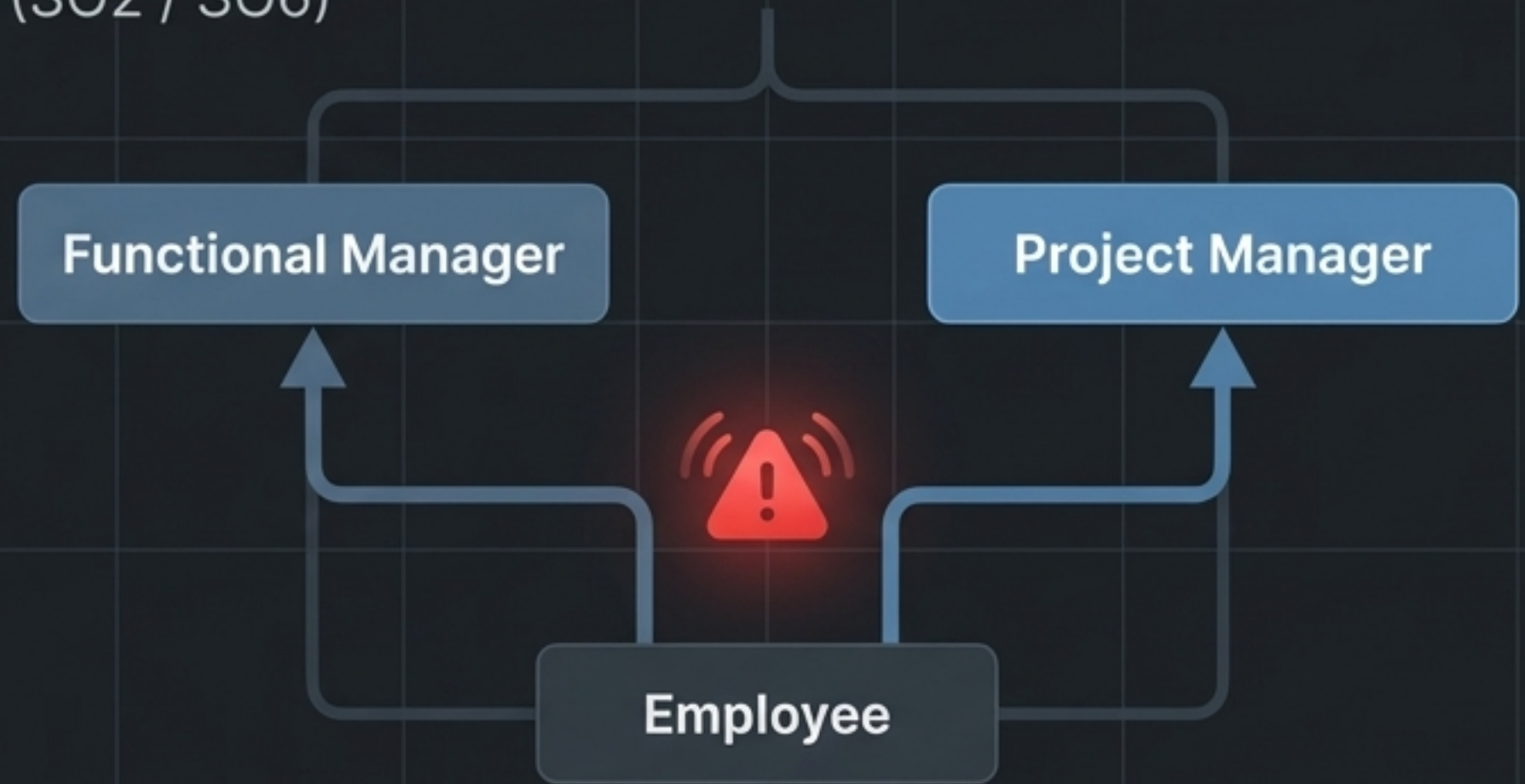
Fixed pipeline. Slow, permanent.

Project-Based Org:

Dynamic teams created per project.

Matrix Organization:

The modern hybrid. People work in a department AND on a project simultaneously.



The Friction Point

The Matrix allows rapid response and specialized skill deployment, but it directly violates Fayol's Principle #4: Unity of Command. PMs often have high responsibility but little formal authority—they must use influence, not just rank.

Domain 2: Engineering the Plan

Planning & Scheduling (SO3)

Earned Value (SO4)

Project Selection (SO5)

Software Estimation (S12)

A plan is not a wish. It is a mathematical model of your project's future. Here we transition from the hierarchy of deliverables (Work Breakdown Structure) into the hard technical skills of scheduling, algorithmic estimation (COCOMO 81, Albrecht's Method), and resource smoothing.

The PERT Probability Engine

Project end-dates are not fixed points. They are probabilities.

The Math of Prediction:

1. Expected Duration:

$$ED = \frac{A + 4M + B}{6}$$

2. Variance on Critical Path:

$$Var = ((B - A) / 6)^2$$

3. The Z-Score:

$$Z = \frac{x - \mu}{s}$$

Start

Expected Duration (ED)

Target End Date

End

Probability
of Success

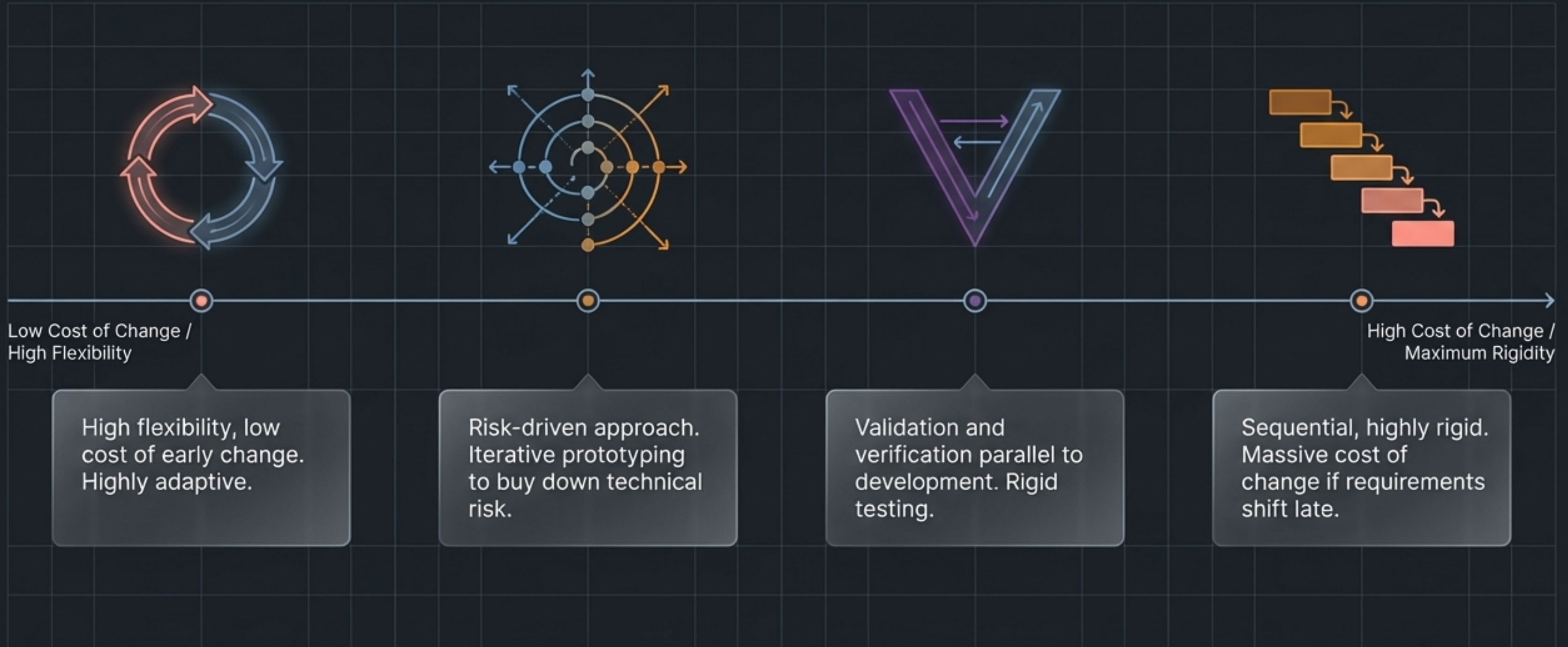
TABLE A Standard Normal Cumulative Probabilities

z	.00	.01	.02	.03	.04	.05	.06	.07	.08	.09
0.0	.5000	.5040	.5080	.5120	.5160	.5199	.5238	.5278	.5319	.5359
0.1	.5298	.5338	.5378	.5417	.5457	.5497	.5537	.5576	.5615	.5655
0.2	.5693	.5732	.5770	.5808	.5846	.5884	.5921	.5958	.5995	.6032
0.3	.6068	.6105	.6143	.6179	.6215	.6251	.6286	.6321	.6355	.6389
0.4	.6423	.6457	.6491	.6524	.6557	.6590	.6622	.6654	.6686	.6718
0.5	.6749	.6780	.6811	.6841	.6871	.6900	.6929	.6958	.6986	.7015
0.6	.7043	.7071	.7098	.7125	.7152	.7179	.7206	.7232	.7258	.7283
0.7	.7308	.7332	.7356	.7379	.7401	.7424	.7445	.7466	.7486	.7507
0.8	.7527	.7547	.7567	.7586	.7605	.7623	.7641	.7658	.7675	.7692
0.9	.7709	.7725	.7742	.7758	.7774	.7789	.7805	.7820	.7835	.7850
1.0	.7865	.7879	.7893	.7907	.7921	.7935	.7948	.7961	.7974	.7987
1.1	.7999	.8011	.8023	.8035	.8046	.8057	.8068	.8078	.8088	.8098
1.2	.8107	.8116	.8125	.8133	.8141	.8149	.8156	.8163	.8170	.8177
1.3	.8184	.8191	.8198	.8205	.8212	.8218	.8224	.8230	.8236	.8241
1.4	.8247	.8252	.8257	.8262	.8267	.8271	.8276	.8280	.8284	.8288
1.5	.8291	.8295	.8299	.8303	.8307	.8310	.8314	.8317	.8320	.8323
1.6	.8325	.8328	.8331	.8334	.8337	.8340	.8343	.8345	.8348	.8350
1.7	.8353	.8355	.8357	.8359	.8361	.8363	.8365	.8367	.8369	.8371
1.8	.8374	.8376	.8378	.8379	.8381	.8383	.8385	.8386	.8388	.8389
1.9	.8391	.8392	.8394	.8395	.8396	.8397	.8398	.8399	.8400	.8401
2.0	.8403	.8404	.8405	.8406	.8407	.8408	.8409	.8409	.8410	.8411
2.1	.8413	.8414	.8415	.8416	.8417	.8418	.8419	.8419	.8420	.8421
2.2	.8421	.8422	.8423	.8424	.8425	.8425	.8426	.8427	.8427	.8428
2.3	.8428	.8429	.8429	.8430	.8431	.8431	.8432	.8432	.8433	.8433
2.4	.8434	.8434	.8435	.8435	.8436	.8436	.8437	.8437	.8438	.8438
2.5	.8438	.8438	.8439	.8439	.8439	.8440	.8440	.8440	.8441	.8441
2.6	.8441	.8441	.8442	.8442	.8442	.8443	.8443	.8443	.8444	.8444
2.7	.8444	.8444	.8445	.8445	.8445	.8446	.8446	.8446	.8447	.8447
2.8	.8447	.8447	.8447	.8448	.8448	.8448	.8448	.8449	.8449	.8449
2.9	.8449	.8449	.8450	.8450	.8450	.8450	.8451	.8451	.8451	.8452
3.0	.8451	.8452	.8452	.8452	.8453	.8453	.8453	.8454	.8454	.8454
3.1	.8454	.8455	.8455	.8455	.8456	.8456	.8456	.8457	.8457	.8457
3.2	.8457	.8457	.8458	.8458	.8458	.8459	.8459	.8459	.8460	.8460
3.3	.8460	.8460	.8461	.8461	.8461	.8462	.8462	.8462	.8463	.8463
3.4	.8463	.8463	.8464	.8464	.8464	.8465	.8465	.8465	.8466	.8466
3.5	.8466	.8466	.8467	.8467	.8467	.8468	.8468	.8468	.8469	.8469
3.6	.8469	.8469	.8470	.8470	.8470	.8471	.8471	.8471	.8472	.8472
3.7	.8472	.8472	.8473	.8473	.8473	.8474	.8474	.8474	.8475	.8475
3.8	.8475	.8475	.8476	.8476	.8476	.8477	.8477	.8477	.8478	.8478
3.9	.8478	.8478	.8479	.8479	.8479	.8480	.8480	.8480	.8481	.8481
4.0	.8481	.8481	.8482	.8482	.8482	.8483	.8483	.8483	.8484	.8484
4.1	.8484	.8484	.8485	.8485	.8485	.8486	.8486	.8486	.8487	.8487
4.2	.8487	.8487	.8488	.8488	.8488	.8489	.8489	.8489	.8490	.8490
4.3	.8490	.8490	.8491	.8491	.8491	.8492	.8492	.8492	.8493	.8493
4.4	.8493	.8493	.8494	.8494	.8494	.8495	.8495	.8495	.8496	.8496
4.5	.8496	.8496	.8497	.8497	.8497	.8498	.8498	.8498	.8499	.8499
4.6	.8499	.8499	.8500	.8500	.8500	.8501	.8501	.8501	.8502	.8502
4.7	.8502	.8502	.8503	.8503	.8503	.8504	.8504	.8504	.8505	.8505
4.8	.8505	.8505	.8506	.8506	.8506	.8507	.8507	.8507	.8508	.8508
4.9	.8508	.8508	.8509	.8509	.8509	.8510	.8510	.8510	.8511	.8511
5.0	.8511	.8512	.8512	.8512	.8513	.8513	.8513	.8514	.8514	.8514

The Reality Check:

We calculate variance and use the Z-table because a timeline is a spectrum of risk. By adding variances along the critical path, we calculate the exact mathematical probability of delivering the software on a specific date.

Domain 3: The SDLC Spectrum Matrix (S07)



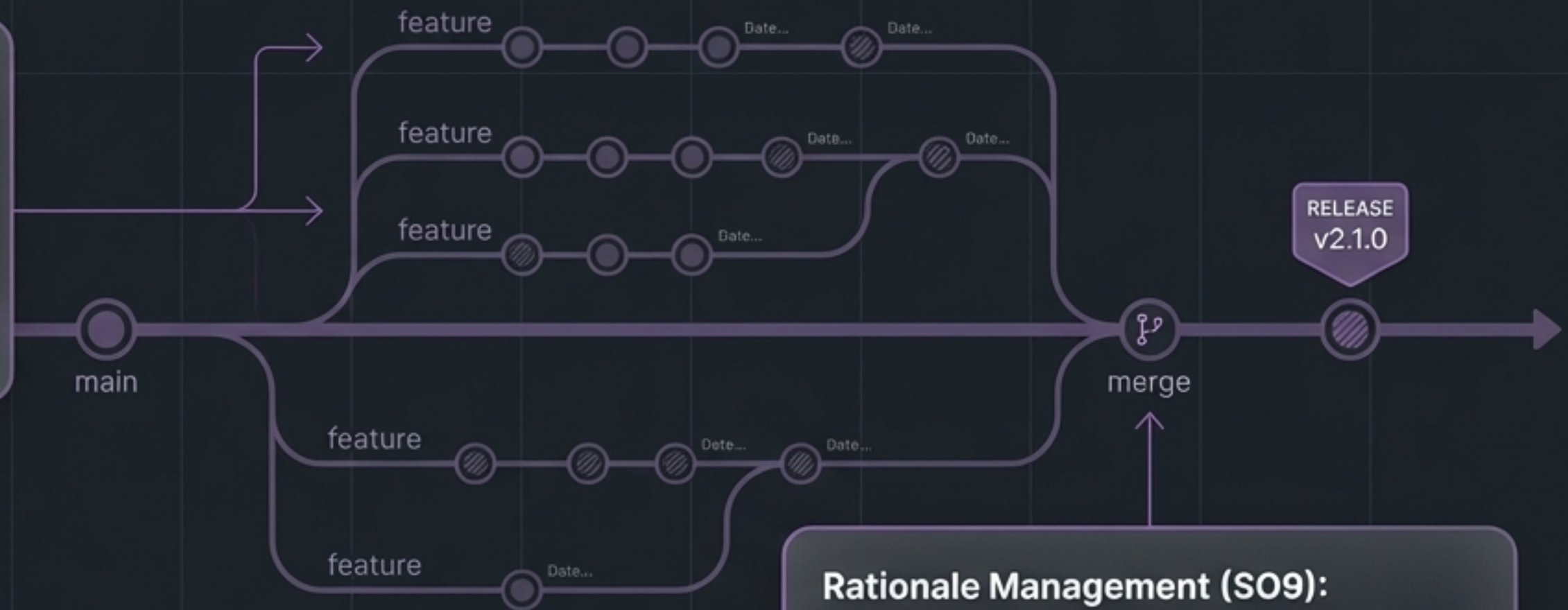
Configuration & Rationale Management

Writing code in a team environment requires engineered alignment.

Configuration Management (SO8):

Controlling the evolution of complex systems.

- **Branch Management:** Isolating feature work to prevent code collisions.
- **Release & Variant Management:** Tracking specific versions deployed to specific environments.



Rationale Management (SO9):

The value of Decision Justification.

Recording **WHY** a technical choice was made, preventing future developers from blindly undoing critical architectural decisions.

Domain 4: Control, Risk & Quality

Execution without monitoring is just hoping for the best.

Earned Value Analysis (S04)

Managing the project baseline. Using metrics to mathematically prove if you are behind schedule or over budget, rather than relying on gut feeling.

Risk Management (S10)

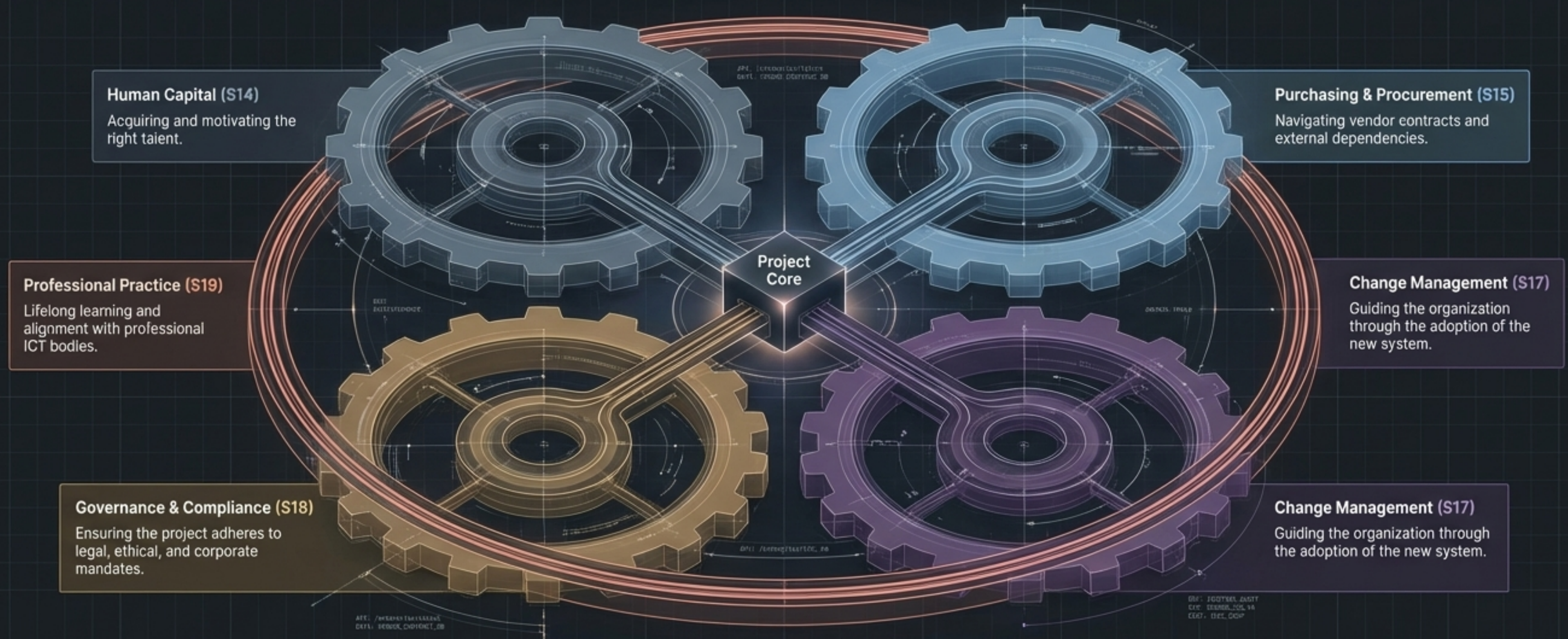
Defining how risk arises and structuring concrete mitigation plans before the impact hits the Iron Triangle.

Quality & Maturity (S13)

Ensuring the organizational capability to deliver. Utilizing ISO standards, Capability Maturity Models (CMM), and Project Audits to institutionalize excellence.

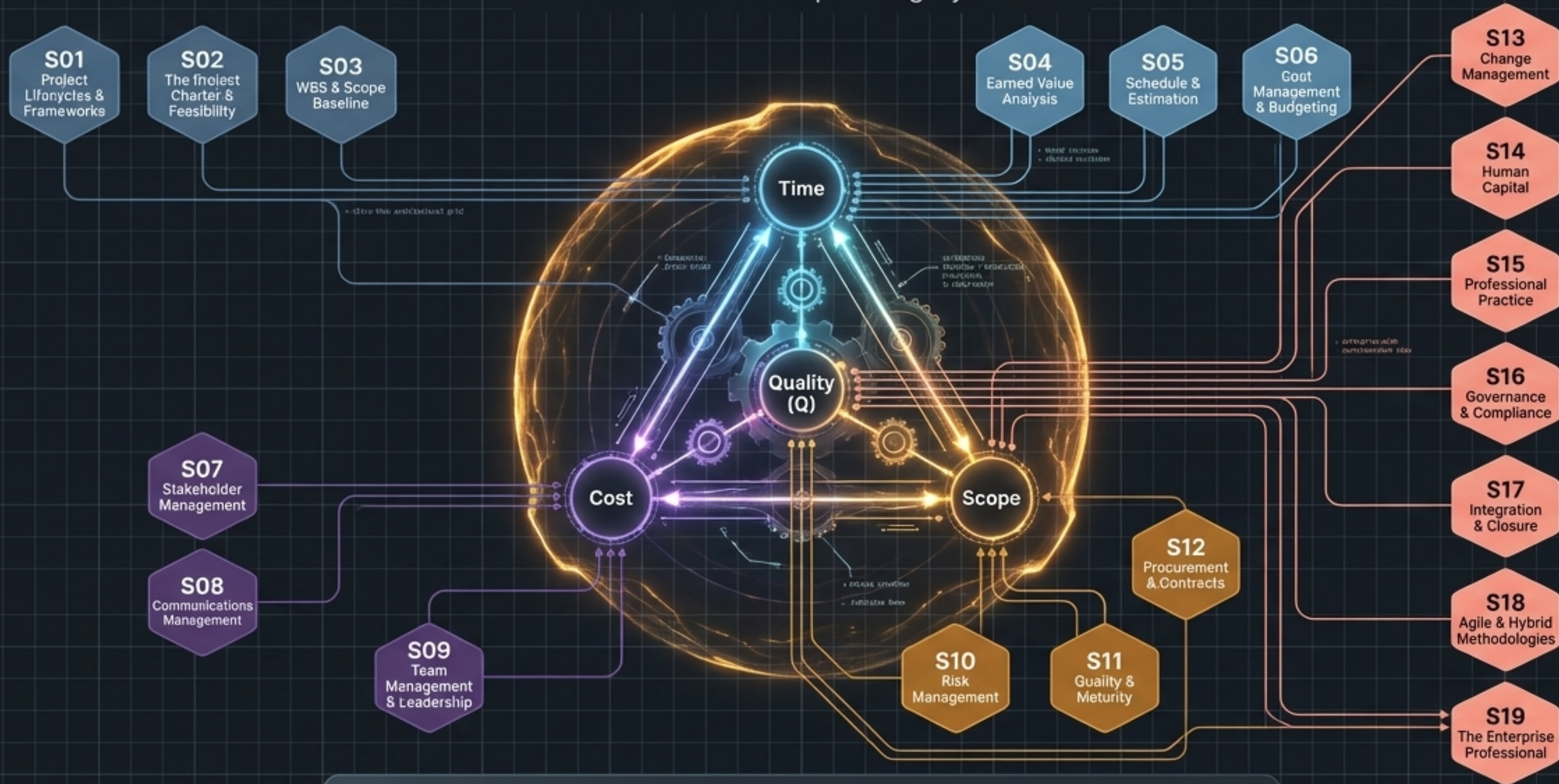
Domain 5: The Enterprise Professional

The final transition. A Senior Project Manager doesn't just manage code; they manage the enterprise ecosystem.



The Master Blueprint

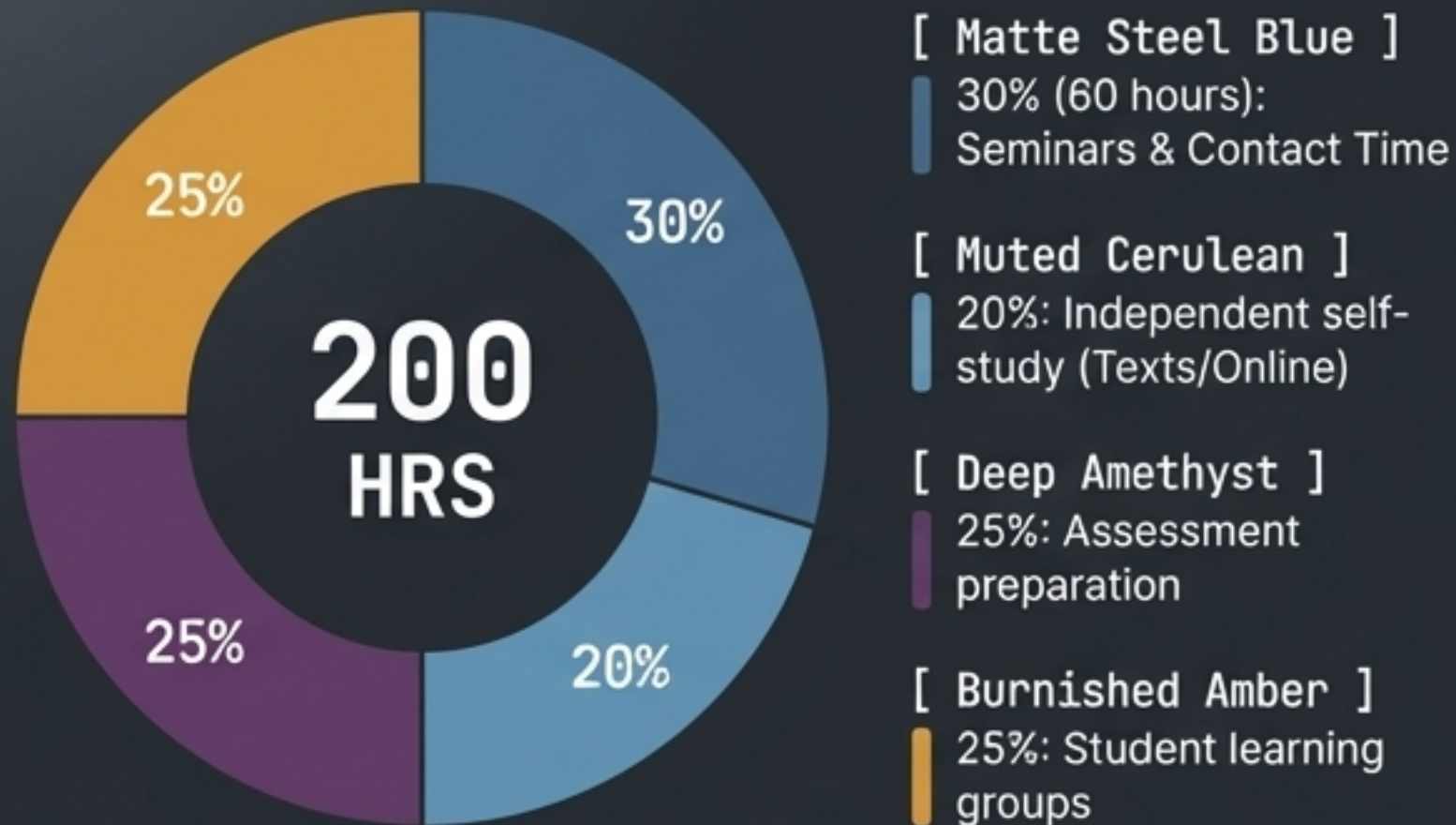
19 Modules. One Operating System.



Every module in this course exists to manage the constraints of the Iron Triangle. You are not studying 19 separate topics. You are mastering a single, unified framework designed to bring chaos under total professional control.

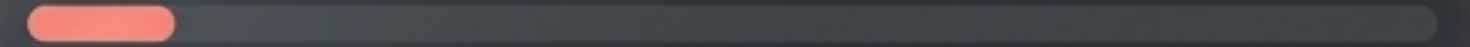
Mission Parameters: PFD473S Logistics

Time Allocation - 200 Notional Hours

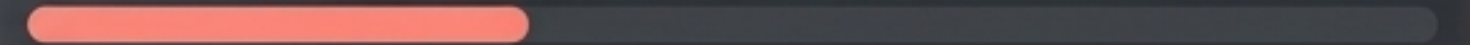


Assessment Weights

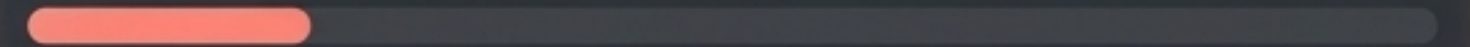
Term 1 (Test 1): 10%



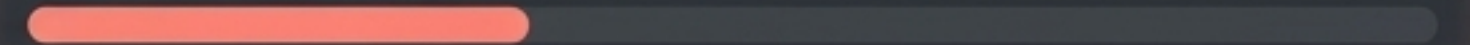
Term 2 (Test 2): 35% (Moderated)



Term 3 (Test 3): 20%



Term 4 (Test 4): 35% (Moderated)



Protocol: NO late submissions are allowed for any subject in this programme. Crims system strictly for non-lecturer queries.

Compile the Professional.

You have the technical competence to build the system.
Now, acquire the architectural vision to lead the enterprise.

```
Review the Subject Guide.  
Log into Blackboard.  
Your transition begins now.  
> _
```